



TRUSTSEC

Smart Contract Audit

Morpho – BlueBundlesV1

26/08/2026

Executive summary

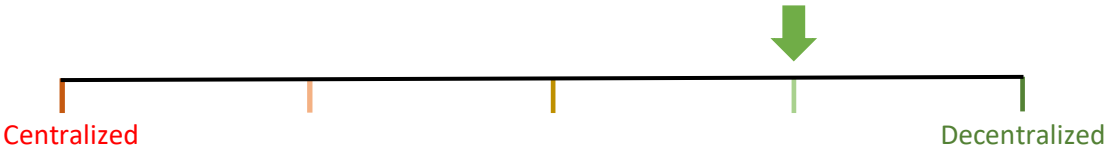


Category	Lending
Audited file count	2
Auditor	HollaDieWaldfee KlosMitSoss
Time period	19/08/2026 - 24/08/2026

Findings

Severity	Total	Fixed	Acknowledged
High	0	0	0
Medium	1	1	0
Low	1	1	0

Centralization score



Signature



Digitally signed by
Trust
Date: 2026.08.26
11:37:58+03'00'

EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	3
Versioning	3
Contact	3
INTRODUCTION	4
Scope	4
Repository details	4
About TrustSec	4
About the Auditors	4
Disclaimer	4
Methodology	5
QUALITATIVE ANALYSIS	6
FINDINGS	7
Medium severity findings	7
TRST-M-1 Static public allocations can charge excessive penalties	7
Low severity findings	9
TRST-L-1 Full debt repayments can be grieved	9
Additional recommendations	10
TRST-R-1 Migration allowance updates do not follow their consumption order	10
TRST-R-2 Empty repay and borrow actions prevent collateral-only operations	11
TRST-R-3 Optional and ignored inputs accept inconsistent calldata	11
TRST-R-4 Inconsistent guidance for disabling the additional LTV check	12
TRST-R-5 Zero returned shares cause arithmetic panics instead of slippage errors	12
TRST-R-6 Share-price checks are unnecessary for validated markets	12
Centralization risks	13
TRST-CR-1 BlueBundlesV1 inherits Morpho Blue centralization risks	13
Systemic risks	14
TRST-SR-1 Supported token assumptions are inherited from Morpho Blue	14
TRST-SR-2 BlueBundlesV1 depends on integrated Morpho contracts	14

Document properties

Versioning

Version	Date	Description
0.1	24/08/2026	Client report
0.2	26/08/2026	Mitigation review

Contact

Trust

trust@trust-security.xyz

Introduction

TrustSec has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

- `src/blue/BlueBundlesV1.sol`
- `src/blue/interfaces/IBlueBundlesV1.sol`

Repository details

- **Repository URL:** <https://github.com/morpho-org/bundles>
- **Commit hash:** `ae0b59ed3e8c57db3a2ff68a85971debf8b5c5a1`
- **Mitigation hash:** `068d625de4623a522b25196928fc4b053a953141`

About TrustSec

TrustSec has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at TrustSec and Senior Watson at Sherlock.

KlosMitSoss is a security researcher with a strong competitive auditing track record, including multiple first place finishes on Code4rena. He has extensive experience conducting both private security reviews and public audit competitions.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. TrustSec assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Excellent	<i>BlueBundlesV1</i> is a minimal and well-structured periphery contract.
Documentation	Good	The project is generally well documented, but some integration behavior and design assumptions could be described more explicitly.
Best practices	Excellent	Project adheres to best practices.
Centralization risks	Excellent	<i>BlueBundlesV1</i> does not introduce additional centralization risks.

Findings

Medium severity findings

TRST-M-1 Static public allocations can charge excessive penalties

- **Category:** Slippage issues
- **Source:** BlueBundlesV1.sol
- **Status:** Fixed

Description

Public allocations and penalties are fixed in calldata, and every submitted allocation executes before the Morpho Blue operation. Front ends are expected to size these allocations, but an off-chain quote only reflects a temporary state snapshot. The live withdrawal output, source debt, or liquidity requirement can decrease before *blueBundlesV1Withdraw()* or *blueBundlesV1MigrateBorrowPosition()* executes without reducing the fixed **PublicAllocations** entries or **penaltyAssets**.

For a share-denominated withdrawal, bad debt realized before execution can reduce the assets represented by the requested shares. *blueBundlesV1Withdraw()* still pays every quoted penalty from the lower output. The penalty may consume most or all the proceeds, or the transaction may revert if the output cannot repay the penalty flash loan.

The same issue exists for borrow migrations. Once the source position is unhealthy, a permissionless liquidator can front-run the migration and profitably repay almost all the source debt while leaving a tiny nonzero remainder. The bundle reads this reduced live debt but executes the original allocations. *onMorphoRepay()* adds the full **penaltyAssets** to the live repaid **assets** when determining destination **borrowAssets**, causing the user to take on the obsolete penalty as new debt. **maxLtv** does not protect against this because liquidation can improve the remaining LTV, and no input bounds the penalty relative to the live debt.

If additional liquidity merely becomes available while the operation amount remains unchanged, a submitted allocation may be unnecessary but the penalty relative to withdrawn or borrowed assets does not increase. This accepted consequence of the static design is distinct from the relative overpayment above and should be documented explicitly.

Recommended mitigation

It is recommended to add user-controlled execution-time bounds, such as a maximum penalty relative to live withdrawn or borrowed assets. On share withdrawals, a bound based on live assets necessarily also constrains share-price movement which currently is [not checked](#) intentionally. This trade-off should be explicit and the bound may be disabled with a sentinel value.

Note that *blueBundlesV1SupplyCollateralAndBorrow()*, which also uses public allocations, is not affected by this issue since it always borrows a specific amount of **borrowAssets**.

Team response

Added documentation in [PR129](#). Rationale: We intentionally execute the user passed PA allocations unconditionally and not dynamically stop the allocations when sufficient liquidity is sourced. With this design, we can not achieve the below two properties simultaneously:

- no shareprice slippage check on withdraw
- slippage check on the ratio of PA penalty fee / net withdrawn Assets

We prefer 1 over 2.

Mitigation review

The finding has been addressed via documentation. It has been determined that it's not the responsibility of *BlueBundlesV1* to dynamically size allocations and users bear the risk of overpaying penalties.

Low severity findings

TRST-L-1 Full debt repayments can be grieved

- **Category:** Griefing issues
- **Source:** BlueBundlesV1.sol
- **Status:** Fixed

Description

blueBundlesV1RepayAndWithdrawCollateral() allows users to repay debt by specifying an exact number of borrow shares. To close a position, the documentation instructs users to pass their full borrow share balance.

Repayments on Morpho Blue are permissionless. An attacker can front-run a full repayment by repaying a small number of shares on behalf of the user. The bundle still forwards the previously quoted share amount:

```
(repayAssets, repayShares) = IMorpho(BLUE).repay(  
    marketParams, repayAssets, repayShares, msg.sender, ""  
);
```

As the requested amount now exceeds the user's remaining borrow shares, Morpho Blue reverts when subtracting it. The user's repayment and collateral withdrawal therefore fail. The attacker can repeat this whenever the user resubmits with an updated exact share amount, at the cost of repaying another small part of the user's debt.

Recommended mitigation

It is recommended to support a sentinel such as **repayShares == type(uint256).max** for full repayments. The bundle should replace the sentinel with the user's live borrow share balance immediately before calling Morpho Blue. If no debt remains, the repayment can be skipped while still allowing the requested collateral withdrawal.

Team response

Fixed in [PR126](#).

Mitigation review

Verified, the recommended mitigation has been implemented.

Additional recommendations

TRST-R-1 Migration allowance updates do not follow their consumption order

During a migration with an allocation penalty, Morpho Blue first pulls the source repayment after *onMorphoRepay()* returns and later pulls the flash-loaned penalty after *onMorphoFlashLoan()* returns. However, the bundle sets the loan-token allowance once before the flash loan and again inside *onMorphoRepay()*. Both instances of setting the allowance therefore occur before either allowance is consumed.

While this behavior is safe for the supported tokens, it is recommended to set the allowance for the flash loan at the end of *onMorphoFlashLoan()*. This ensures that each token transfer is executed with a fresh allowance.

The required change is shown below:

```

--- a/src/blue/BlueBundlesV1.sol
+++ b/src/blue/BlueBundlesV1.sol
@@ -88,7 +88,6 @@
         executeBorrow(marketParams, borrowAssets, minSharePriceE27,
            reallocations, msg.sender);
     } else {
         bytes memory operationData = abi.encode(marketParams, borrowAssets,
            minSharePriceE27, reallocations);
-         TokenLib.forceApproveMax(marketParams.loanToken, BLUE);
         IMorpho(BLUE)
             .flashLoan(
                 marketParams.loanToken,
@@ -229,7 +228,6 @@
         executeWithdraw(marketParams, withdrawAssets, withdrawShares,
            reallocations, msg.sender);
     } else {
         bytes memory operationData = abi.encode(marketParams, withdrawAssets,
            withdrawShares, reallocations);
-         TokenLib.forceApproveMax(marketParams.loanToken, BLUE);
         IMorpho(BLUE)
             .flashLoan(
                 marketParams.loanToken,
@@ -312,7 +310,6 @@
         bytes memory operationData = abi.encode(
            sourceMarketParams, uint256(position.borrowShares),
            sourceMaxSharePriceE27, reallocations, migrationData
        );
-         TokenLib.forceApproveMax(destMarketParams.loanToken, BLUE);
         IMorpho(BLUE)
             .flashLoan(
                 destMarketParams.loanToken,
@@ -382,6 +379,7 @@
         PublicAllocations[] memory reallocations
            = abi.decode(operationData, (MarketParams, uint256, uint256,
            PublicAllocations[]));
         executeBorrow(marketParams, borrowAssets, minSharePriceE27,
            reallocations, sender);
+         TokenLib.forceApproveMax(marketParams.loanToken, BLUE);
     } else if (selector == this.blueBundlesV1Withdraw.selector) {
         (
            MarketParams memory marketParams,
@@ -390,6 +388,7 @@
         PublicAllocations[] memory reallocations
            = abi.decode(operationData, (MarketParams, uint256, uint256,
            PublicAllocations[]));
         executeWithdraw(marketParams, withdrawAssets, withdrawShares,
            reallocations, sender);
+         TokenLib.forceApproveMax(marketParams.loanToken, BLUE);
     }

```

```

    } else if (selector == this.blueBundlesV1MigrateBorrowPosition.selector) {
        (
            MarketParams memory sourceMarketParams,
@@ -407,6 +406,7 @@
            penaltyAssets,
            sender
        );
+       TokenLib.forceApproveMax(sourceMarketParams.loanToken, BLUE);
    } else {
        revert UnauthorizedCallback();
    }
}

```

TRST-R-2 Empty repay and borrow actions prevent collateral-only operations

blueBundlesV1SupplyCollateralAndBorrow() always calls Morpho Blue's borrow function, even when **borrowAssets** is zero. Similarly, *blueBundlesV1RepayAndWithdrawCollateral()* calls the repay function when both repayment inputs are zero. Morpho Blue requires exactly one of the asset and share inputs to be zero, so the empty action reverts and rolls back the useful collateral action.

As a result, the composite entry points cannot be used to supply collateral without borrowing or withdraw collateral without repaying.

It is recommended to skip the borrow or repay call when that leg is empty and another requested collateral action remains. Authorization and resulting LTV checks should still be applied where relevant.

TRST-R-3 Optional and ignored inputs accept inconsistent calldata

Some inputs are ignored depending on the selected mode. This does not create a direct security impact, but silently accepted fields can hide malformed calldata or an unintended integration path.

When **PublicAllocations.fromIdle** is true, *executePublicAllocations()* calls *allocateFromIdle()* and ignores **sourceAdapter** and **sourceMarketParams**, as documented by the interface. A nonzero ignored source adapter has no effect but can indicate that an integration selected the wrong allocation mode.

Similarly, *setAuthorizationWithSig()* treats zero-valued **v**, **r**, and **s** as the absence of an optional authorization and ignores nonzero **nonce** and **deadline**. This is not a replay or authorization bypass, but an unintended mix of an empty signature and populated metadata is silently accepted.

If canonical encodings are desired, require **sourceAdapter == address(0)** for **fromIdle=true** allocations and require **nonce** and **deadline** to be zero with an empty signature.

TRST-R-4 Inconsistent guidance for disabling the additional LTV check

The additional maximum-LTV check is skipped whenever `maxLtv >= marketParams.lltv`. However, the [NatSpec](#) for `blueBundlesV1SupplyCollateralAndBorrow()` recommends **WAD** to disable the check, while the migration [NatSpec](#) recommends the destination market LLTV. Both values work, but using different conventions for the same behavior is error-prone.

It is recommended to use one convention throughout the codebase and document the actual condition: a value at or above the relevant market LLTV disables only the bundle-level bound, while Morpho Blue's normal health check remains enforced.

TRST-R-5 Zero returned shares cause arithmetic panics instead of slippage errors

Morpho Blue rounds down when converting a positive asset-denominated supply or repayment to shares. At a sufficiently high asset-per-share ratio, the returned **suppliedShares** or **repayShares** can be zero. The bundle then uses the returned value as the denominator of `mulDivUp()`:

```
require(toSupply.mulDivUp(1e27, suppliedShares) <= maxSharePriceE27,
  SlippageExceeded());
```

`mulDivUp()` evaluates `d - 1`, where `d` is the denominator. A zero denominator therefore causes **Panic(0x11)** before the intended **SlippageExceeded()** error can be raised and callers receive an opaque arithmetic error for a slippage condition.

It is recommended to explicitly require that **suppliedShares** and **repayShares** are nonzero before calculating the share price and to revert with **SlippageExceeded()** or another dedicated error when the conversion returns zero.

TRST-R-6 Share-price checks are unnecessary for validated markets

The share-price checks in *BlueBundlesV1* protect users that interact with markets before their balances have been safely seeded. Morpho applications already verify that a [dead deposit](#) has been made before enabling a market. Given that markets are validated, the share-price parameters add complexity to the entry points without benefit for security.

It is recommended to remove the share-price parameters if *BlueBundlesV1* is intended only for markets that pass this eligibility check. Otherwise, retain and document the checks for direct callers that may interact with arbitrary or unseeded markets.

Centralization risks

TRST-CR-1 BlueBundlesV1 inherits Morpho Blue centralization risks

BlueBundlesV1 does not introduce any centralization risks of its own. The contract has no privileged roles or administrative functions. However, users inherit the centralization risks of Morpho Blue and, when public allocations are used, Morpho *VaultV2* and *BluePublicAllocator*.

Systemic risks

TRST-SR-1 Supported token assumptions are inherited from Morpho Blue

BlueBundlesV1 uses the loan and collateral tokens configured in Morpho Blue markets and inherits [Morpho Blue's token safety requirements](#). Supported tokens must also tolerate the [approval sequence](#) `approve(..., 0)` followed by `approve(..., type(uint256).max)`. Tokens outside the documented support assumptions should not be treated as supported merely because the bundle functions accept their addresses.

TRST-SR-2 BlueBundlesV1 depends on integrated Morpho contracts

BlueBundlesV1 is a periphery contract that routes user operations through Morpho Blue. Operations that request public allocations additionally depend on *BluePublicAllocator*, *VaultV2*, and the vault's adapters. An exploit, accounting failure, or loss of availability in any involved dependency can cause bundle calls to execute incorrectly, revert, or expose users to losses. *BlueBundlesV1* cannot isolate users from failures in those contracts.